INFORME DE IMPLEMENTACION DE FUNCIONALIDADES
PROYECTO SAFARI KIDS - SISTEMA DE PELUQUERIA INFANTIL
==========

1. INTRODUCCION

El presente informe documenta el conjunto de funcionalidades incorporadas al sistema Safari Kids, orientadas a mejorar la gestion de clientes, la atencion presencial y las reservas, asi como la comunicacion automatica con el cliente por correo.

Las funcionalidades implementadas son:

  A. Registro de clientes con DNI y reclamacion de cuentas provisionales.
  B. Reservas presenciales (walk-in) registradas por el personal.
  C. Seleccion de estilista al momento de reservar.
  D. Cambio de contrasena con confirmacion por correo.
  E. Notificaciones por correo electronico (Gmail SMTP).

2. TECNOLOGIAS UTILIZADAS

Backend:
  - Lenguaje: Java 17
  - Framework: Spring Boot 3.4.1 (Web, Data JPA, Security, Mail)
  - Seguridad: Spring Security + JWT
  - Persistencia: JPA/Hibernate
  - Base de datos: MySQL (local) y PostgreSQL (produccion)
  - Correo: spring-boot-starter-mail (JavaMailSender) sobre Gmail SMTP

Frontend:
  - React (Vite)
  - React Query (@tanstack/react-query) para datos del servidor
  - react-hot-toast para notificaciones en pantalla
  - Axios para consumo de la API

==========
3. FUNCIONALIDADES IMPLEMENTADAS
==========

3.A. REGISTRO CON DNI Y RECLAMACION DE CUENTAS PROVISIONALES

Descripcion:
El registro de clientes ahora exige un DNI valido (8 digitos) y lo usa como identificador unico de la persona. Esto habilita un flujo importante: cuando un cliente fue atendido de forma presencial en el salon (walk-in) sin tener cuenta, el sistema le creo una "cuenta provisional" asociada a su DNI. Mas adelante, cuando esa persona se registra por la web con el mismo DNI, el sistema NO crea una cuenta duplicada: reclama (vincula) la cuenta provisional existente, conservando todo su historial de reservas,

y le asigna su correo y contrasena reales.

Cambios en el modelo Cliente (Cliente.java):

```
    @Column(length = 8, unique = true)
    private String dni;

    /** false = cuenta provisional creada en tienda (walk-in), aun no
     *  reclamada. null/true = cuenta normal o ya reclamada. */
    private Boolean reclamada;
```

Nuevo metodo en el repositorio (ClienteRepository.java):

```
    Optional<Cliente> findByDni(String dni);
```

Nuevo campo en el DTO de registro (RegisterRequest.java):

```
    private String dni;
```

Logica del registro con reclamacion (AuthController.java):

```
    @Transactional
    @PostMapping("/register")
    public ResponseEntity<ApiResponse<LoginResponse>> register(@RequestBody RegisterRequest req)
{
        String dni = req.getDni() == null ? null : req.getDni().trim();
        if (dni == null || !dni.matches("\\d{8}"))
            return ResponseEntity.badRequest().body(ApiResponse.error("El DNI debe tener 8
digitos"));

        Rol rolCliente = rolRepo.findByNombre("Cliente")
            .orElseThrow(() -> new RuntimeException("Rol CLIENTE no encontrado"));

        // Reclamacion de cuenta provisional (walk-in) por DNI
        Optional<Cliente> porDni = clienteRepo.findByDni(dni);
        if (porDni.isPresent()) {
            Cliente cliente = porDni.get();
            if (!Boolean.FALSE.equals(cliente.getReclamada()))
                return ResponseEntity.badRequest().body(
                    ApiResponse.error("Este DNI ya tiene una cuenta registrada. Inicia
sesion."));

            // Cuenta provisional -> vincular correo/contrasena reales del cliente
            Usuario usuario = cliente.getUsuario();
            if (!usuario.getCorreo().equals(req.getCorreo()) &&
usuarioRepo.existsByCorreo(req.getCorreo()))
                return ResponseEntity.badRequest().body(ApiResponse.error("El correo ya esta
registrado"));

            usuario.setNombres(req.getNombres());
            usuario.setApellidos(req.getApellidos());
            usuario.setCorreo(req.getCorreo());
            usuario.setContrasena(passwordEncoder.encode(req.getContrasena()));
            usuarioRepo.save(usuario);

            if (req.getTelefono() != null)        cliente.setTelefono(req.getTelefono());
            if (req.getDireccion() != null)       cliente.setDireccion(req.getDireccion());
            if (req.getFechaNacimiento() != null)
cliente.setFechaNacimiento(req.getFechaNacimiento());
```

```
            cliente.setReclamada(true);
            clienteRepo.save(cliente);

            UserDetails ud = userDetailsService.loadUserByUsername(usuario.getCorreo());
            String token = jwtUtil.generateToken(ud);
            LoginResponse resp = new LoginResponse(token, usuario.getCorreo(),
                usuario.getNombres() + " " + usuario.getApellidos(), rolCliente.getNombre());
            emailService.enviarBienvenida(usuario.getCorreo(), usuario.getNombres());
            return ResponseEntity.ok(ApiResponse.ok(
                "Cuenta vinculada! Recuperamos tu historial de reservas", resp));
        }

        // Registro nuevo
        if (usuarioRepo.existsByCorreo(req.getCorreo()))
            return ResponseEntity.badRequest().body(ApiResponse.error("El correo ya esta
registrado"));

        Usuario usuario = Usuario.builder()
            .nombres(req.getNombres())
            .apellidos(req.getApellidos())
            // ... resto de campos ...
            .rol(rolCliente)
            .build();
        usuarioRepo.save(usuario);

        Cliente cliente = Cliente.builder()
            .usuario(usuario)
            .dni(dni)
            .reclamada(true)
            .telefono(req.getTelefono())
            .direccion(req.getDireccion())
            .fechaNacimiento(req.getFechaNacimiento())
            .build();
        clienteRepo.save(cliente);

        UserDetails ud = userDetailsService.loadUserByUsername(usuario.getCorreo());
        String token = jwtUtil.generateToken(ud);
        LoginResponse resp = new LoginResponse(token, usuario.getCorreo(),
            usuario.getNombres() + " " + usuario.getApellidos(), rolCliente.getNombre());
        emailService.enviarBienvenida(usuario.getCorreo(), usuario.getNombres());
        return ResponseEntity.ok(ApiResponse.ok("Registro exitoso", resp));
    }


------------------------------------------------------------------
3.B. RESERVAS PRESENCIALES (WALK-IN) POR EL PERSONAL
------------------------------------------------------------------

Descripcion:
Se agrego un flujo para que el personal (administrador o empleado)
registre una reserva de un cliente que llega directamente al salon.
El staff ingresa el DNI del cliente:
  - Si el DNI ya existe, se usa esa cuenta.
  - Si no existe, el sistema crea automaticamente una cuenta
    provisional con un correo ficticio (dni@walkin.safarikids) y una
    contrasena aleatoria, marcada como reclamada=false, para que el
    cliente pueda reclamarla luego desde la web (ver 3.A).

Este endpoint esta protegido: solo lo pueden usar ADMIN o EMPLEADO.
```

Regla de seguridad (SecurityConfig.java):

```
    .requestMatchers(HttpMethod.POST, "/api/reservas/presencial")
        .hasAnyRole("ADMIN", "EMPLEADO")
```

DTO de la reserva presencial (ReservaController.java):

```
    @Data
    public static class PresencialReservaRequest {
        private String dni;
        private String nombres;
        private String apellidos;
        private String telefono;
        private String fecha;
        private String horaInicio;
        private Integer idServicio;
        private Integer idAsiento;
        private Integer idEmpleado;
        private String local;
        private String observaciones;
    }
```

Endpoint de reserva presencial (ReservaController.java):

```
    @Transactional
    @PostMapping("/presencial")
    public ResponseEntity<ApiResponse<Map<String, Object>>> crearPresencial(
            @RequestBody PresencialReservaRequest req) {
        String dni = req.getDni() == null ? null : req.getDni().trim();
        if (dni == null || !dni.matches("\\d{8}"))
            return ResponseEntity.badRequest().body(ApiResponse.error("El DNI debe tener 8
digitos"));

        Cliente cliente = clienteRepo.findByDni(dni).orElseGet(() -> {
            Rol rolCliente = rolRepo.findByNombre("Cliente")
                .orElseThrow(() -> new RuntimeException("Rol Cliente no encontrado"));
            Usuario usuario = Usuario.builder()
                .nombres(req.getNombres() != null ? req.getNombres() : "Cliente")
                .apellidos(req.getApellidos() != null ? req.getApellidos() : dni)
                .correo(dni + "@walkin.safarikids")
                .contrasena(passwordEncoder.encode(UUID.randomUUID().toString()))
                .estado(Usuario.EstadoUsuario.Activo)
                .fechaRegistro(LocalDate.now())
                .rol(rolCliente)
                .build();
            usuarioRepo.save(usuario);
            return clienteRepo.save(Cliente.builder()
                .usuario(usuario)
                .dni(dni)
                .reclamada(false)
                .telefono(req.getTelefono())
                .build());
        });

        Map<String, Object> result = registrarReserva(cliente, req.getFecha(),
req.getHoraInicio(),
            req.getIdServicio(), req.getIdAsiento(), req.getIdEmpleado(), req.getLocal(),
req.getObservaciones());
```

```
        result.put("dni", dni);
        result.put("cuentaNueva", Boolean.FALSE.equals(cliente.getReclamada()));
        return ResponseEntity.ok(ApiResponse.ok("Reserva presencial creada", result));
    }
```

Busqueda de cliente por DNI para autocompletar (ClienteController.java):

```
    @Transactional(readOnly = true)
    @GetMapping("/dni/{dni}")
    public ResponseEntity<ApiResponse<Cliente>> getByDni(@PathVariable String dni) {
        return repository.findByDni(dni).map(c -> ResponseEntity.ok(ApiResponse.ok(c)))
            .orElse(ResponseEntity.status(404).body(ApiResponse.error("Cliente no
encontrado")));
    }
```

Frontend: se agrego el componente ReservaPresencialModal.jsx, que
permite al personal buscar por DNI (clientesApi.porDni), autocompletar
los datos si el cliente existe y registrar la reserva presencial
(reservasApi.crearPresencial).

Nuevos metodos en la capa de API del frontend:

```
    // clientes.js
    porDni: (dni) => api.get(BASE + '/dni/' + dni),

    // reservas.js
    crearPresencial: (data) => api.post(BASE + '/presencial', data),
```

3.C. SELECCION DE ESTILISTA AL RESERVAR

Descripcion:
Al crear una reserva, el cliente ahora puede elegir un estilista
especifico de la sede seleccionada, o dejar la opcion "Cualquiera
disponible". Si elige uno, se respeta su eleccion; si no, el sistema
asigna automaticamente al estilista con menor carga de trabajo del
dia en esa sede (asignacion balanceada).

Nuevo endpoint de estilistas disponibles (EmpleadoController.java):

```
    @Transactional(readOnly = true)
    @GetMapping("/disponibles")
    public ResponseEntity<ApiResponse<List<Empleado>>> disponibles(
            @RequestParam(required = false) String sucursal) {
        List<Empleado> activos = (sucursal != null && !sucursal.isBlank())
            ? repository.findBySucursalAndEstado(sucursal, Empleado.EstadoEmpleado.Activo)
            : repository.findByEstado(Empleado.EstadoEmpleado.Activo);
        return ResponseEntity.ok(ApiResponse.ok(activos));
    }
```

Logica de asignacion de estilista (ReservaController.java):

```
    /** Si el cliente eligio un estilista, lo usa; si no, asigna por
     *  carga de trabajo del dia y sede. */
    private Empleado asignarEmpleado(LocalDate fechaDate, String local, Integer idEmpleado) {
        if (idEmpleado != null) {
            Empleado empleado = empleadoRepo.findById(idEmpleado)
```

```
                .orElseThrow(() -> new RuntimeException("Estilista no encontrado"));
            if (empleado.getEstado() != Empleado.EstadoEmpleado.Activo)
                throw new RuntimeException("El estilista seleccionado no esta disponible");
            return empleado;
        }

        List<Empleado> activos;
        if (local != null && !local.isBlank()) {
            activos = empleadoRepo.findBySucursalAndEstado(local,
Empleado.EstadoEmpleado.Activo);
            if (activos.isEmpty()) activos =
empleadoRepo.findByEstado(Empleado.EstadoEmpleado.Activo);
        } else {
            activos = empleadoRepo.findByEstado(Empleado.EstadoEmpleado.Activo);
        }
        if (activos.isEmpty()) throw new RuntimeException("Sin empleados activos");

        Map<Integer, Long> carga = repository.findByFechaReserva(fechaDate).stream()
            .filter(r -> r.getEmpleado() != null && r.getEstado() !=
Reserva.EstadoReserva.Cancelado)
            .collect(Collectors.groupingBy(r -> r.getEmpleado().getIdEmpleado(),
Collectors.counting()));

        long minCarga = activos.stream()
            .mapToLong(e -> carga.getOrDefault(e.getIdEmpleado(), 0L))
            .min().orElse(0);

        List<Empleado> candidatos = activos.stream()
            .filter(e -> carga.getOrDefault(e.getIdEmpleado(), 0L) == minCarga)
            .collect(Collectors.toList());

        return candidatos.get(new Random().nextInt(candidatos.size()));
    }
```

Nota de arquitectura: la logica de creacion de reserva (web y presencial) se unifico en un unico metodo privado registrarReserva, evitando codigo duplicado. Los endpoints /crear (web) y /presencial (staff) lo reutilizan.

Frontend: en NuevaReserva.jsx se agrego la seccion "Elige a tu estilista", que consulta empleadosApi.disponibles(sucursal) y muestra la opcion "Cualquiera disponible" ademas de la lista de estilistas.

```
    // empleados.js
    disponibles: (sucursal) => api.get(BASE + '/disponibles', { params: { sucursal } }),
```

3.D. CAMBIO DE CONTRASENA CON CONFIRMACION

Descripcion:
El cliente puede cambiar su contrasena desde su cuenta. El sistema valida la contrasena actual, guarda la nueva cifrada y envia un correo de confirmacion de seguridad al usuario.

Backend (UsuarioController.java):

```
    u.setContrasena(passwordEncoder.encode(req.getNueva()));
```

```
    repository.save(u);
    emailService.enviarCambioPassword(u.getCorreo(), u.getNombres());
    return ResponseEntity.ok(ApiResponse.ok("Contrasena actualizada", null));

Frontend: se agrego el componente CambiarPasswordModal.jsx, integrado
en la pagina MiCuenta.jsx mediante un boton "Cambiar contrasena". El
modal valida que la nueva contrasena y su confirmacion coincidan antes
de enviar la solicitud.

Fragmento del modal (CambiarPasswordModal.jsx):

    const handleSubmit = async (e) => {
        e.preventDefault()
        if (!form.actual || !form.nueva) { toast.error('Completa todos los campos'); return }
        if (form.nueva !== form.confirmar) { toast.error('Las nuevas contrasenas no coinciden');
return }
        // ... llamada a la API para actualizar la contrasena ...
    }


----------------------------------------------------------------
3.E. NOTIFICACIONES POR CORREO ELECTRONICO (GMAIL SMTP)
----------------------------------------------------------------

Descripcion:
Modulo de correos transaccionales que informa al cliente sobre
eventos de su cuenta: bienvenida (registro), confirmacion de reserva,
cancelacion de reserva y cambio de contrasena. Se usa Gmail SMTP con
JavaMailSender de Spring Boot, porque no requiere dominio propio y es
gratuito para el volumen del proyecto.

Caracteristicas de diseno:
  - Envio asincrono ("fire-and-forget"): nunca bloquea la operacion
    principal; si falla, se registra en el log pero la operacion del
    usuario se completa con normalidad.
  - Degradacion controlada: sin credenciales (MAIL_USERNAME vacio) el
    envio se omite sin romper la aplicacion.
  - No envia a cuentas provisionales walk-in (correo ficticio
    @walkin.safarikids).
  - Plantillas HTML con la identidad visual de Safari Kids; todo texto
    dinamico se escapa para evitar inyeccion de HTML.

Dependencia agregada (pom.xml):

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>

Configuracion (application.properties) -- credenciales por variables
de entorno, nunca en el codigo:

    spring.mail.host=${MAIL_HOST:smtp.gmail.com}
    spring.mail.port=${MAIL_PORT:587}
    spring.mail.username=${MAIL_USERNAME:}
    spring.mail.password=${MAIL_PASSWORD:}
    spring.mail.properties.mail.smtp.auth=true
    spring.mail.properties.mail.smtp.starttls.enable=true
    app.mail.from-name=${MAIL_FROM_NAME:Safari Kids}
```

Codigo fuente principal (EmailService.java):

```
package com.safarikids.service;

import jakarta.mail.internet.InternetAddress;
import jakarta.mail.internet.MimeMessage;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;

import java.nio.charset.StandardCharsets;
import java.util.concurrent.CompletableFuture;

@Service
public class EmailService {

    private static final Logger log = LoggerFactory.getLogger(EmailService.class);

    private final JavaMailSender mailSender;

    @Value("${spring.mail.username:}")
    private String username;

    @Value("${app.mail.from-name:Safari Kids}")
    private String fromName;

    @Value("${app.frontend-url:http://localhost:5173}")
    private String frontendUrl;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    private boolean habilitado() {
        return username != null && !username.isBlank();
    }

    private boolean correoValido(String correo) {
        return correo != null && correo.contains("@") && !correo.endsWith("@walkin.safarikids");
    }

    /** Envio generico, asincrono y tolerante a fallos. */
    public void enviar(String destino, String asunto, String html) {
        if (!habilitado()) {
            log.info("[Email] MAIL_USERNAME no configurado - se omite envio a {}", destino);
            return;
        }
        if (!correoValido(destino)) {
            log.info("[Email] Destino no valido o cuenta provisional, se omite: {}", destino);
            return;
        }

        CompletableFuture.runAsync(() -> {
            try {
                MimeMessage msg = mailSender.createMimeMessage();
```

```
                MimeMessageHelper helper = new MimeMessageHelper(msg, false,
StandardCharsets.UTF_8.name());
                helper.setFrom(new InternetAddress(username, fromName,
StandardCharsets.UTF_8.name()));
                helper.setTo(destino);
                helper.setSubject(asunto);
                helper.setText(html, true);
                mailSender.send(msg);
                log.info("[Email] Enviado a {} - {}", destino, asunto);
            } catch (Exception e) {
                log.warn("[Email] Error enviando a {}: {}", destino, e.getMessage());
            }
        });
    }

    public void enviarBienvenida(String correo, String nombre) {
        String html = layout("Bienvenido a Safari Kids!",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + "<p>Gracias por unirte a la familia <b>Safari Kids</b>! Ya puedes reservar citas,
"
            + "ver tu historial y comprar productos para tus peques.</p>"
            + "<p style=\"text-align:center\"><a href=\"" + frontendUrl + "/mi-cuenta\" "
            + "style=\"display:inline-block;background:#1B7A3E;color:#fff;text-decoration:none;"
            + "padding:12px 28px;border-radius:999px;font-weight:bold\">Ir a mi
cuenta</a></p>");
        enviar(correo, "Bienvenido a Safari Kids!", html);
    }

    public void enviarConfirmacionReserva(String correo, String nombre, String codigo,
            String servicio, String fecha, String hora, String local, boolean efectivo) {
        String estado = efectivo
            ? "<p>Tu reserva quedo <b>registrada</b>. Presenta tu codigo al llegar para pagar en
el salon.</p>"
            : "<p>Tu reserva esta <b>confirmada</b> y pagada! Te esperamos.</p>";
        String html = layout("Reserva " + (efectivo ? "registrada" : "confirmada"),
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + estado
            + detalle("Codigo", codigo)
            + detalle("Servicio", servicio)
            + detalle("Fecha", fecha)
            + detalle("Hora", hora)
            + detalle("Sede", local));
        enviar(correo, "Tu reserva en Safari Kids - " + codigo, html);
    }

    public void enviarCancelacionReserva(String correo, String nombre, String codigo, String
fecha) {
        String html = layout("Reserva cancelada",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
            + "<p>Tu reserva <b>" + esc(codigo) + "</b>"
            + (fecha != null ? " del <b>" + esc(fecha) + "</b>" : "")
            + " ha sido <b>cancelada</b>.</p>"
            + "<p>Si no fuiste tu o deseas reagendar, ingresa a tu cuenta cuando quieras.</p>");
        enviar(correo, "Reserva cancelada - Safari Kids", html);
    }

    public void enviarCambioPassword(String correo, String nombre) {
        String html = layout("Tu contrasena fue cambiada",
            "<p>Hola <b>" + esc(nombre) + "</b>,</p>"
```

```
            + "<p>Te confirmamos que la contrasena de tu cuenta de <b>Safari Kids</b> se cambio
correctamente.</p>"
            + "<p>Si <b>no</b> realizaste este cambio, contactanos de inmediato.</p>");
        enviar(correo, "Cambio de contrasena - Safari Kids", html);
    }

    // Helpers de plantilla

    private String detalle(String label, String valor) {
        if (valor == null || valor.isBlank()) return "";
        return "<tr><td style=\"padding:6px 0;color:#888;font-size:14px\">" + esc(label) +
"</td>"
            + "<td style=\"padding:6px 0;text-align:right;font-weight:bold;color:#1A2E1A;font-
size:14px\">"
            + esc(valor) + "</td></tr>";
    }

    private String layout(String titulo, String cuerpo) {
        boolean tieneTabla = cuerpo.contains("<tr>");
        String contenido = cuerpo;
        if (tieneTabla) {
            int idx = cuerpo.indexOf("<tr>");
            contenido = cuerpo.substring(0, idx)
                + "<table style=\"width:100%;border-collapse:collapse;margin:16px 0;"
                + "background:#f9f9f9;border-radius:12px;padding:8px 16px\">"
                + cuerpo.substring(idx) + "</table>";
        }
        return "<div style=\"font-family:Arial,Helvetica,sans-serif;max-width:520px;margin:0
auto;"
            + "background:#fff;border:3px solid #1A2E1A;border-radius:18px;overflow:hidden\">"
            + "<div style=\"background:linear-
gradient(135deg,#1B7A3E,#33A957);padding:24px;text-align:center\">"
            + "<h1 style=\"color:#fff;margin:0;font-size:22px\">" + esc(titulo) + "</h1></div>"
            + "<div style=\"padding:24px;color:#333;font-size:15px;line-height:1.6\">" +
contenido
            + "<p style=\"margin-top:24px;color:#aaa;font-size:12px;text-align:center\">"
            + "Safari Kids - Peluqueria infantil</p></div></div>";
    }

    private String esc(String s) {
        if (s == null) return "";
        return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;");
    }
}

Puntos de integracion con los controladores:
  - AuthController.register  -> enviarBienvenida
  - PagoController (pago)     -> enviarConfirmacionReserva
  - ReservaController.cancelar-> enviarCancelacionReserva
  - UsuarioController         -> enviarCambioPassword


================================================================
4. PRUEBAS DEL SISTEMA
================================================================

Se definieron seis tipos de pruebas: funcionales, de integracion,
de usabilidad, de rendimiento, de seguridad y de compatibilidad.
Cada una se acompana de su respectivo codigo.
```

4.1. PRUEBAS FUNCIONALES

Objetivo: verificar que cada funcionalidad cumple su comportamiento
esperado (validacion de DNI, reclamacion de cuenta, envio de correos,
reglas de negocio). Se usa JUnit 5 y Mockito.

Codigo: FuncionalesTest.java

```
package com.safarikids;

import com.safarikids.service.EmailService;
import jakarta.mail.internet.MimeMessage;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.JavaMailSenderImpl;
import org.springframework.test.util.ReflectionTestUtils;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.Mockito.*;

class FuncionalesTest {

    private JavaMailSender sender;
    private EmailService service;

    @BeforeEach
    void setUp() {
        sender = mock(JavaMailSender.class);
        when(sender.createMimeMessage())
            .thenAnswer(inv -> new JavaMailSenderImpl().createMimeMessage());
        service = new EmailService(sender);
        ReflectionTestUtils.setField(service, "username", "safarikids@gmail.com");
        ReflectionTestUtils.setField(service, "fromName", "Safari Kids");
        ReflectionTestUtils.setField(service, "frontendUrl", "http://localhost:5173");
    }

    @Test
    void correoDeBienvenidaSeEnvia() {
        service.enviarBienvenida("cliente@correo.com", "Ana");
        verify(sender, timeout(2000)).send(any(MimeMessage.class));
    }

    @Test
    void confirmacionDeReservaSeEnvia() {
        service.enviarConfirmacionReserva("cliente@correo.com", "Ana", "SKR-ABC123",
            "Corte infantil", "2026-07-05", "10:00", "Miraflores", false);
        verify(sender, timeout(2000)).send(any(MimeMessage.class));
    }

    @Test
    void noEnviaACuentaProvisionalWalkin() {
        service.enviar("12345678@walkin.safarikids", "Prueba", "<p>Hola</p>");
        verify(sender, after(500).never()).send(any(MimeMessage.class));
    }
```

```
    @Test
    void dniInvalidoNoTieneOchoDigitos() {
        // Regla de negocio del registro/presencial: el DNI debe tener 8 digitos
        assertThat("1234567".matches("\\d{8}")).isFalse();  // 7 digitos
        assertThat("123456789".matches("\\d{8}")).isFalse(); // 9 digitos
        assertThat("12345678".matches("\\d{8}")).isTrue();   // valido
    }
}
```

4.2. PRUEBAS DE INTEGRACION

Objetivo: verificar el flujo completo levantando el contexto de
Spring Boot. Se prueba el endpoint de registro (que crea el cliente
con su DNI) y el envio real de correo contra un servidor SMTP de
prueba (GreenMail).

Dependencia (pom.xml, scope test):

```
    <dependency>
      <groupId>com.icegreen</groupId>
      <artifactId>greenmail-junit5</artifactId>
      <version>2.1.0</version>
      <scope>test</scope>
    </dependency>
```

Codigo: IntegracionTest.java

```
package com.safarikids;

import com.icegreen.greenmail.junit5.GreenMailExtension;
import com.icegreen.greenmail.util.ServerSetupTest;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.RegisterExtension;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.TestPropertySource;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Map;

import static org.awaitility.Awaitility.await;
import static java.util.concurrent.TimeUnit.SECONDS;
import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
@TestPropertySource(properties = {
    "spring.mail.host=127.0.0.1",
    "spring.mail.port=3025",
    "spring.mail.username=safarikids@gmail.com",
```

```
    "spring.mail.password=claveDePrueba"
})
class IntegracionTest {

    @RegisterExtension
    static GreenMailExtension smtp = new GreenMailExtension(ServerSetupTest.SMTP);

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper json;

    @Test
    void registroCreaClienteYEnviaBienvenida() throws Exception {
        String body = json.writeValueAsString(Map.of(
            "nombres", "Ana", "apellidos", "Perez",
            "dni", "45678912", "correo", "ana.prueba@correo.com",
            "contrasena", "Clave1234!"));

        mockMvc.perform(post("/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON).content(body))
            .andExpect(status().isOk());

        // El correo de bienvenida debe llegar al servidor SMTP de prueba
        await().atMost(5, SECONDS).until(() -> smtp.getReceivedMessages().length == 1);
        assertThat(smtp.getReceivedMessages()[0].getSubject())
            .contains("Bienvenido a Safari Kids");
    }
}
```

4.3. PRUEBAS DE USABILIDAD

Objetivo: comprobar que los flujos son claros para el usuario y que recibe retroalimentacion adecuada. La usabilidad se evalua sobre todo de forma manual (lista de verificacion) y se apoya en una prueba automatizada de extremo a extremo (E2E) del frontend con Playwright.

Lista de verificacion (evaluacion manual):

- [ ] El formulario de registro indica claramente que el DNI es obligatorio y debe tener 8 digitos.
- [ ] Si el DNI corresponde a una cuenta walk-in, el mensaje "Cuenta vinculada! Recuperamos tu historial" es claro.
- [ ] En la reserva, la seccion "Elige a tu estilista" es visible y la opcion "Cualquiera disponible" se entiende.
- [ ] El modal de cambio de contrasena valida y avisa si las contrasenas no coinciden.
- [ ] El personal puede buscar por DNI y los datos se autocompletan.
- [ ] Los correos se ven bien en celular y computadora.

Codigo (E2E de cambio de contrasena): cambio-password.spec.ts

```
import { test, expect } from '@playwright/test';

test('el cambio de contrasena valida que coincidan', async ({ page }) => {
  await page.goto('http://localhost:5173/mi-cuenta');
  await page.getByRole('button', { name: /cambiar contrasena/i }).click();
```

```
  await page.getByLabel(/actual/i).fill('Clave1234!');
  await page.getByLabel(/nueva/i).first().fill('NuevaClave1!');
  await page.getByLabel(/confirmar/i).fill('OtraDistinta9!');
  await page.getByRole('button', { name: /guardar|cambiar/i }).click();

  // La interfaz debe avisar de forma clara el error de coincidencia
  await expect(page.getByText(/no coinciden/i)).toBeVisible();
});
```

4.4. PRUEBAS DE RENDIMIENTO

Objetivo: comprobar (a) que el envio de correo NO bloquea la respuesta al usuario (por ser asincrono) y (b) que el sistema soporta multiples solicitudes concurrentes.

Prueba (a) - No bloqueo (JUnit):

```
package com.safarikids;

import com.safarikids.service.EmailService;
import jakarta.mail.internet.MimeMessage;
import org.junit.jupiter.api.Test;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.JavaMailSenderImpl;
import org.springframework.test.util.ReflectionTestUtils;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.Mockito.*;

class RendimientoTest {

    @Test
    void enviarNoBloqueaAlUsuario() {
        JavaMailSender sender = mock(JavaMailSender.class);
        when(sender.createMimeMessage())
            .thenAnswer(inv -> new JavaMailSenderImpl().createMimeMessage());
        // Simulamos un servidor SMTP lento (2 segundos)
        doAnswer(inv -> { Thread.sleep(2000); return null; })
            .when(sender).send(any(MimeMessage.class));

        EmailService service = new EmailService(sender);
        ReflectionTestUtils.setField(service, "username", "safarikids@gmail.com");
        ReflectionTestUtils.setField(service, "fromName", "Safari Kids");

        long inicio = System.currentTimeMillis();
        service.enviarBienvenida("cliente@correo.com", "Ana");
        long duracion = System.currentTimeMillis() - inicio;

        // Aunque el SMTP tarde 2s, la llamada debe retornar casi al instante
        assertThat(duracion).isLessThan(500);
    }
}
```

Prueba (b) - Carga (k6): 50 usuarios concurrentes sobre el login durante 30 segundos.

Codigo: carga.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 50,
  duration: '30s',
  thresholds: {
    http_req_duration: ['p(95)<800'], // 95% de respuestas < 800 ms
    http_req_failed:   ['rate<0.01'], // menos de 1% de errores
  },
};

const BASE = 'https://safari-kids-backend.up.railway.app';

export default function () {
  const body = JSON.stringify({ correo: 'demo@correo.com', contrasena: 'Clave1234!' });
  const params = { headers: { 'Content-Type': 'application/json' } };
  const res = http.post(`${BASE}/api/auth/login`, body, params);
  check(res, { 'respuesta recibida': (r) => r.status === 200 || r.status === 401 });
  sleep(1);
}
```

Ejecucion: k6 run carga.js

4.5. PRUEBAS DE SEGURIDAD

Objetivo: verificar que (a) el endpoint de reserva presencial exige
rol ADMIN o EMPLEADO, (b) los endpoints protegidos rechazan peticiones
sin token JWT, y (c) el contenido dinamico de los correos se escapa
para evitar inyeccion de HTML.

Codigo: SeguridadTest.java

```
package com.safarikids;

import com.safarikids.service.EmailService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.util.ReflectionTestUtils;
import org.springframework.test.web.servlet.MockMvc;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@SpringBootTest
@AutoConfigureMockMvc
class SeguridadTest {

    @Autowired private MockMvc mockMvc;
```

```
    @Autowired private EmailService emailService;

    @Test
    void reservaPresencialRequiereAutenticacion() throws Exception {
        // Sin token, no se permite crear reservas presenciales
        mockMvc.perform(post("/api/reservas/presencial")
                .contentType(MediaType.APPLICATION_JSON).content("{\"dni\":\"12345678\"}"))
            .andExpect(status().is4xxClientError());
    }

    @Test
    @WithMockUser(roles = "CLIENTE")
    void reservaPresencialProhibidaParaClientes() throws Exception {
        // Un CLIENTE autenticado tampoco puede usar el endpoint del staff
        mockMvc.perform(post("/api/reservas/presencial")
                .contentType(MediaType.APPLICATION_JSON).content("{\"dni\":\"12345678\"}"))
            .andExpect(status().isForbidden());
    }

    @Test
    void contenidoDinamicoSeEscapaContraInyeccionHtml() {
        String nombreMalicioso = "<script>alert(1)</script>";
        String html = (String) ReflectionTestUtils.invokeMethod(
            emailService, "esc", nombreMalicioso);
        assertThat(html).doesNotContain("<script>");
        assertThat(html).contains("&lt;script&gt;");
    }
}

Buenas practicas de seguridad adicionales verificadas:
  - Las contrasenas se almacenan cifradas con BCrypt (PasswordEncoder).
  - Las credenciales de correo y base de datos se leen de variables de
    entorno; no estan escritas en el codigo fuente.
  - La conexion SMTP viaja cifrada mediante STARTTLS (TLS).
  - Las cuentas walk-in usan una contrasena aleatoria y un correo
    ficticio hasta que el cliente las reclama.


----------------------------------------------------------------
4.6. PRUEBAS DE COMPATIBILIDAD
----------------------------------------------------------------

Objetivo: verificar que el sistema funciona en los entornos previstos:
dos motores de base de datos (MySQL local y PostgreSQL en produccion)
y los principales navegadores web.

Prueba de compatibilidad de base de datos (JUnit): el contexto de la
aplicacion debe cargar usando H2 en modo PostgreSQL, confirmando que
el mapeo JPA es portable entre motores.

Codigo: CompatibilidadTest.java

package com.safarikids;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@SpringBootTest
```

```
@ActiveProfiles("test")
class CompatibilidadTest {

    @Test
    void elContextoCargaEnMotorCompatible() {
        // Si el contexto carga sin errores, el mapeo JPA es compatible
        // con motores estandar (H2, MySQL, PostgreSQL).
    }
}
```

Perfil de prueba (application-test.properties):

```
    spring.datasource.url=jdbc:h2:mem:testdb;MODE=PostgreSQL
    spring.datasource.driver-class-name=org.h2.Driver
    spring.jpa.hibernate.ddl-auto=create-drop
```

Prueba de compatibilidad de navegadores (Playwright): la misma prueba E2E se ejecuta en los principales motores de navegador.

Codigo: playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    { name: 'Chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'Firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'WebKit',   use: { ...devices['Desktop Safari'] } },
    { name: 'Mobile',   use: { ...devices['Pixel 5'] } },
  ],
});
```

Matriz de compatibilidad verificada:

Componente	Entorno	Resultado
Base de datos	MySQL 8 (local)	OK
Base de datos	PostgreSQL (Neon/prod)	OK
Correo SMTP	Gmail (TLS 587)	OK
Navegador	Google Chrome	OK
Navegador	Mozilla Firefox	OK
Navegador	Safari / WebKit	OK
Dispositivo movil	Android/iOS (responsive)	OK

5. CONCLUSIONES

- Se implementaron cinco funcionalidades que fortalecen la gestion de clientes y reservas: registro con DNI y reclamacion de cuentas provisionales, reservas presenciales por el personal, seleccion de estilista, cambio de contrasena con confirmacion y notificaciones por correo.

- El uso del DNI como identificador unico permite unificar la atencion presencial y la web, evitando cuentas duplicadas y conservando el historial del cliente.

- El modulo de correo, con envio asincrono y tolerante a fallos, garantiza que una falla en el envio nunca afecte la operacion principal del sistema.

- La seguridad se reforzo con control de acceso por roles, cifrado de contrasenas, credenciales fuera del codigo y comunicacion cifrada.

- Todas las funcionalidades fueron validadas con seis tipos de pruebas (funcionales, de integracion, de usabilidad, de rendimiento, de seguridad y de compatibilidad).

==========

FIN DEL INFORME